

features:

- 1) markdown syntax.
- 2) version control. - easy reversion
- 3) collab editing
- 4) easy navigation.

Setting:

- 1) Creating:
 - (repo - github)
 - wiki - 1st page
 - title, markdown.
 - save
- 2) cloning: (locally)
 - git clone ..url..

structure:

- installation instructions
- usage guides
- API documentation
- FAQs

Advanced:

① Collaboration:

- (i) centralized workflows: - everyone works on single repo
- single branch (main)
- (ii) Feature branch workflow: - each feature - own branch
main with mul branches.
- (iii) Workflow workflow: - structured branching for scheduled releases
- separate prod, dev, testing, hotfix
for periodical releases.

② Tagging: [bookmarks on commit, hash a3f2...X tag v1.0.0]

(log) flow: add, cnt, tag, push main & push tag.

commands:

viewing: \$git tag → all tags.

\$git tag -l "v1.*" → only v1.x tags

\$git show v1.0.1 → cnt + msg behind that tag.

tag old cnt: \$git tag -a v1.0.1 <hash> -m "msg"

delete: \$ git tag -d v1.0.1 (local)
 \$ git push origin --delete v1.0.1 (remote)

checkout: \$ git checkout <tag> — git detached head (not in any branch)
 \$ git checkout -b hotfix/v1.0.0 v1.0.0

push: \$ git push origin --tags
 — new branch start from that tag.
 Next tag may be 1.0.2 - push it.

How sample: checkout -b .. , commit , tag , push <branch> , push tag.

③ cherry picking:

[pick one spec unit from a branch, apply it to urs]
 (eg flow): \$ git log feature-br —online
 : o/p... cnts

\$ git checkout main

single: \$ git cherry-pick <needed-unt's-hash>

multiple unts: \$ git cherry-pick <unt1> <unt2> <unt3> ...

range of unts: \$ git cherry-pick <1st unt> .. <last unt>
 ↳ order matters.
 ↓ exclusive ↓ inclusive

conflicts? fix, add, \$ git cherry-pick --continue /
 --abort

⇒ same commit but made twice,
 so diff hash, twice in history.

④ Git Packfiles:-

prob: - every commit - store snapshot as "loose obj" in
 - as unt ↑ , loose files ↑ → slow & waste.
 - .git/objects/..

soln: packfiles → bundles all loose objects into compressed file
 → stores only differences b/w versions (delta)

how in .git/objects/

a3/f2121c... - loose obj 1
 b7/c3de... - loose obj 2

pack/

pack-abc123.pack - packfile (bundled & compressed)
 pack-abc123.idx - index fr obj inside pack.

when created:-

→ by default by git when pushed ^(send), done ^(receive)
threshold (~6700)

autopacks.

→ explicitly:

↳ garbage collect, pack + cleans

↓ git gc
↓ git gc --aggressive

→ manual operations

(pack) \$git pack-objects --all .git/objects/pack/pack

(verify) \$git verify-pack -v .git/objects/pack/pack-objects.idx

(see) \$git verify-pack -v (↓) 1 head -20

↳ o/p: hash | type | size | comp size

Delta trick:-

- without pack - all lines of file saved again & again

with pack - all lines once, changes alone as verifying

(eg) 1000 lines 1 file

50 commits, 2 lines changed each time

without : $50 \times 1000 = 50000$ lines on disk

with : $1000 + 49 \times 2 (\text{diffs}) = 1098$ lines.
(base)

when done is slow:-

git gc --aggressive

git repack (-a) (-d) --depth=250 --window=250

repack
everything
into one

delete old
loose obj
after

how hard tries to find
deltas - higher = small pack,
slow.

5. Git hooks:

- scripts that run automatically when git events happen.
- line in `.git/hooks/`

⑤

client side

(run on ur machine)

- pre-commit → bef int created
- commit-msg → after type commit msg
- pre-push → bef push

↳ use husky fr manage this in JS projects. → npm install ..

→ configured mostly by shell scripts.

server side

(run on git server)

- pre-receive → bef all push
- post-receive → aft all push
- update → per branch version of pre-receive

↳ webhooks - configured with it.

6. Git worktree :

- ① checkout multiple branches simultaneously.
- ② in same repo, multiple branches.

(add) `$ git worktree add ../branch fin/v1.1.1`

(list) `$ git worktree list`

(remove) `$ git worktree remove ../branch`

7. git subtree & submodules:-

embed another repo.

no .gitmodules, no ptr,

fst merged. (copied)

→ proj depends on another repo.
- embed in ur. (linked)

`$ git submodule add git-url.com/folder`
↳ update, remove, done operations

8. Git filter-branch:

→ rewrites git history. (slow & old)
rem/change across all commits.

Git filter-repo:

pip install git-filter-repo (safe & fast)

→ used when accidentally committed secrets.

(eg): remove file from all history

rename folder across all history